SVM FLEET (RAIDER APP)

Comprehensive System Documentation, Implementation History & Technical Architecture

VERSION 1.0.43
HYDERABAD FLEET OS

Date: November 2026

Executive Overview: SVM FLEET is an enterprise dual-interface fleet operating system purpose-built for commercial fleet operators in Hyderabad managing cabs across **Uber, Rapido, and Ola**. The system unites real-time background accessibility telemetry (zero driver typing), automated fare capture, dual-store data persistence (Room SQLite + Supabase Cloud PostgreSQL), live GPS fleet tracking, a digital Office Duty Register replacing physical logbooks, automated 70/30 commission & fuel accounting, and instant A4 PDF / CSV statements.

1. Complete Implementation Breakdown (What Was Built)

A. Brand Identity & Intro Carousel (Figma Mockups Sheet 1 & 2)

- **Electric Purple (#5B36F5) Adaptive App Icon:** Vector launcher foreground with centered crisp white vehicle emblem on electric purple background.
- **Intro Splash Screen Carousel:** Deep Night purple gradient (#130A3A), perspective animated road lines, pulsing vehicle emblem, 3-dot pagination slider (● ■ ■), and primary 'Enter Fleet Manager' CTA.
- **Unified Design System:** Custom theme palette (Electric Purple, Emerald Green #00C853, Inactive Red #E53935, Surface GlassCards, Pill-shaped controls).

B. Authentication & Dual Access Gateway

- **Dual-Mode Tabs:** Switch seamlessly between Fleet Owner Portal and Driver Shift Companion.
- **Owner Security Master PIN:** Authorized phone authentication + editable Owner Secret Key (Default: 7777).
- **In-App Secure OTP & Fast2SMS:** Instant in-app 4-digit code generator + Fast2SMS / WhatsApp gateway integration.
- **1-Tap Showcase Demo Mode:** Immediate presentation mode with 30 cars, 28 active, live metrics, and historical logs.

C. Driver Shift Companion & Automated Telemetry Engine

- **3-Step Setup Wizard:** Guides driver through Battery Exemption (Unrestricted 24/7 run), Accessibility Service toggle, and DPDP Act 2023 Consent.
- **Silent Ride Ingestion (RideTrackerAccessibilityService):** Captures fares, distance (km), duration, and pickups/dropoffs from Uber Driver and Rapido Captain without driver manual entry.
- **DynamicSelectorEngine & RideStateMachine:** Self-healing dynamic UI tree parser with regex fallback for Uber, Rapido, and Ola screens, ensuring zero ride loss.

D. Dual-Storage & Cloud Realtime Architecture

- **Offline-First Local SQLite (Room):** Tables for vehicles, drivers, rides, ride events, duty logs, and audit trails.
- **Supabase Realtime Cloud Sync:** Integrated postgrest-kt and realtime-kt upserting rides and duty logs directly to PostgreSQL with automatic reconnection and channel subscription ('fleet_telemetry_live').

E. Office Duty Register & Commission Accounting

- **Digital Logbook (OfficeRegisterScreen):** Replaces traditional Moosapet paper logbooks with digital shift logging.
- **SvmCommissionCalculator:** Automatic calculation of opening/closing km, fuel deduction, daily commission/rent, and driver net settlement.
- **Shift Rotation & Driver Handoff:** Car-level shift handover modal (Day Shift 06 AM - 06 PM vs Night Shift 06 PM - 06 AM) with starting odometer validation.

F. Live Fleet Tracking & Interactive Map

- **Custom Canvas Map (LiveTrackingScreen):** Stylized Hyderabad street network (Banjara Hills, Jubilee Hills, Hitec City) with animated radar pulses, route trails, and active/idle car markers.
- **Floating Status Card:** Selected vehicle details (TS09 AB 1234 • On Trip • 8.4 km away), live speed (42 km/h), current fare, and one-tap driver dialer.

G. Document Vault & Native Statement Export

• **Digital Document Vault (DocumentVaultScreen):** Digital compliance cards for RC, Commercial Insurance (ICICI Lombard), PUC, Fitness, and State Permits.

• **A4 Financial Statement Generator:** Programmatic PDF generator using Android PdfDocument drawing purple header banners, owner breakdown, and tabular car metrics.

• **RFC4180 CSV & FileProvider:** Full CSV export shared securely via Android FileProvider to WhatsApp, Drive, Gmail, or Excel.

2. How It Works (End-to-End Pipeline & Architecture)

The SVM FLEET platform operates through a coordinated sequence of distributed background services, local state machines, and real-time cloud channels. The system lifecycle is divided into 5 distinct operational stages:

Stage	Trigger / Process	Underlying Component	Result
1. Driver Onboarding	Owner creates driver profile in Owner Portal with DL, phone, and car assignment	DriverRepository.kt, VehicleRepository.kt	Synced to Room DB & Supabase 'drivers'
2. Shift Start	Driver logs in with OTP, selects assigned vehicle, and taps 'Start Shift'	LoginViewModel.kt, ShiftManager.kt	Shift state set to ACTIVE. Foreground ser
3. Silent Ingestion	Driver accepts and completes trips in Uber/Rapido apps	RideTrackerAccessibilityService.kt, DynamicSelectorEngine.kt	Accessibility tree parsed. Fares (■), KM, p
4. Validation & State	State machine verifies ride transition (OFFERED → ACCEPTED → COMPLETED)	RideStateMachine.kt, RideDao.kt (Room)	Ride stored in local SQLite cache. Immed
5. Cloud Sync & Sub	Supabase real-time manager pushes trip & duty logs to cloud	SupabaseRealtimeSyncManager.kt, DutyLogRepository.kt	Owner Dashboard updates in real time; P

Security, Fault-Tolerance & Regulatory Safeguards

- **DPDP Act 2023 Compliance:** Telemetry is strictly restricted to commercial taxi platforms (com.ubercab.driver, com.rapido.captain, com.olacabs.driver). Personal messaging (WhatsApp), banking, and browser screens are mathematically ignored by package whitelist filters.
- **Zero-Loss Offline Caching:** If network connectivity drops in transit, trips and duty logs remain in Room SQLite. The WorkManager 'OutboxSyncWorker' automatically batches and flushes pending records upon network restoration.
- **Aggressive Task-Killer Defense:** Android 12–15 aggressive OEM power-savers are bypassed via `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS` and sticky foreground service execution with Wakelocks.
- **Owner Master PIN Protection:** Direct in-app secret PIN management enables changing hub access credentials dynamically without re-compiling the APK.

3. Codebase Architectural Tree — Part 1 (Data & Service Layer)

```
raider-app/
■■■ app/
■   ■■■ src/main/
■   ■   ■■■ AndroidManifest.xml                    # Permissions, Accessibility Service & FileProvider
■   ■   ■■■ java/com/fleettrack/app/
■   ■       ■■■ FleetTrackApp.kt                   # Hilt Application entry point & WorkManager scheduler
■   ■       ■■■ MainActivity.kt                    # Single-Activity host, Jetpack Compose navigation
■   ■       ■
■   ■       ■■■ data/                              # Data Architecture (Offline Room + Remote Supabase)
■   ■       ■   ■■■ local/
■   ■       ■   ■   ■■■ FleetDatabase.kt           # Room Database with RideDao, DriverDao, DutyLogDao
■   ■       ■   ■   ■■■ FleetDataSeeder.kt         # Comprehensive 30-car fleet & historical demo seeder
■   ■       ■   ■   ■■■ dao/
■   ■       ■   ■   ■   ■■■ DutyLogDao.kt          # Office logbook entries, cash, fuel, commission queries
■   ■       ■   ■   ■   ■■■ DriverDao.kt           # Driver records and shift assignment DAO
■   ■       ■   ■   ■   ■■■ RideDao.kt             # Captured trip records and sync status DAO
■   ■       ■   ■   ■   ■■■ VehicleDao.kt          # Fleet vehicles and active status DAO
■   ■       ■   ■   ■■■ entity/
■   ■       ■   ■       ■■■ DriverEntity.kt        # Driver phone, license, fleet ID schema
■   ■       ■   ■       ■■■ DutyLogEntity.kt       # Opening/closing km, fuel, rent, cash balance schema
■   ■       ■   ■       ■■■ RideEntity.kt          # Fare, platform, duration, coordinates, status schema
■   ■       ■   ■       ■■■ VehicleEntity.kt       # Reg number, nickname, compliance documents schema
■   ■       ■   ■■■ model/                         # Domain models: DriverPerformance, FleetOverview, VehicleStatus
■   ■       ■   ■■■ remote/                        # Cloud Sync (Supabase Postgrest & WebSockets)
■   ■       ■   ■   ■■■ SupabaseClientProvider.kt  # Supabase client singleton (Realtime & Postgrest)
■   ■       ■   ■   ■■■ SupabaseRealtimeSyncManager.kt # Realtime channel 'fleet_telemetry_live' listener
■   ■       ■   ■   ■■■ RideRemoteDataSource.kt    # Postgrest ride upsert with offline fallback
■   ■       ■   ■   ■■■ DeviceRemoteDataSource.kt  # Heartbeat telemetry and battery reporting
■   ■       ■   ■■■ repository/                    # Clean Architecture Repositories
■   ■       ■   ■   ■■■ DriverRepository.kt        # Driver CRUD, duplicate phone check, cloud sync
■   ■       ■   ■   ■■■ DutyLogRepository.kt       # Office register duty logs & financial settlements
■   ■       ■   ■   ■■■ RideRepository.kt          # Trip queries, aggregation, pending outbox sync
■   ■       ■   ■   ■■■ VehicleRepository.kt       # Vehicle status, fleet metrics, document vault
■   ■       ■   ■■■ session/
■   ■       ■       ■■■ ShiftManager.kt            # Active driver shift state & local preferences
■   ■       ■
■   ■       ■■■ di/                                # Dependency Injection (Hilt / Dagger)
■   ■       ■   ■■■ AppModule.kt                   # Database, network, and application bindings
■   ■       ■   ■■■ RepositoryModule.kt            # Repository interfaces to implementation bindings
■   ■       ■
■   ■       ■■■ service/                           # Telemetry Engine & Screen Scraping
■   ■       ■   ■■■ RideTrackerAccessibilityService.kt # Android Accessibility Service for Uber/Rapido/Ola
■   ■       ■   ■■■ parser/
■   ■       ■   ■   ■■■ DynamicSelectorEngine.kt   # Self-healing dynamic UI tree parser
■   ■       ■   ■   ■■■ AccessibilityNodeUtils.kt  # Node hierarchy traversal & text extractors
■   ■       ■   ■   ■■■ UberDriverParser.kt        # Uber fare, distance, pickup/dropoff regex parser
■   ■       ■   ■   ■■■ RapidoCaptainParser.kt     # Rapido order fare & route extractor
■   ■       ■   ■   ■■■ OlaDriverParser.kt         # Ola driver ride card parser
■   ■       ■   ■■■ statemachine/
■   ■       ■       ■■■ RideStateMachine.kt        # State machine (OFFER -> ACCEPTED -> ON_TRIP -> COMPLETED)
```

3. Codebase Architectural Tree — Part 2 (UI, Utilities & Root)

```
│       ├── ui/                                  # Jetpack Compose UI Presentation Layer
│       │   ├── auth/                            # LoginScreen (Dual Tab, OTP, Secret Key), LoginViewModel
│       │   ├── intro/                           # IntroSplashScreen (3-dot carousel, Deep Night gradient)
│       │   ├── navigation/                      # FleetNavigation, Screen routes, BottomNavBar
│       │   ├── driver/                          # Driver Shift Companion (Today, SetupWizard, Privacy)
│       │   ├── debug/                           # NodeInspectorScreen (Realtime accessibility tree viewer)
│       │   ├── owner/                           # Fleet Owner Portal Modules
│       │   │   ├── dashboard/                   # DashboardScreen (Fleet overview, earnings, Active 28)
│       │   │   ├── cars/                        # CarsListScreen (30 cars, filters, AddCarDialog)
│       │   │   ├── detail/                      # VehicleDetailScreen (Overview, Trips, Earnings, Handoff)
│       │   │   ├── drivers/                     # DriversScreen, DriverDetailScreen (P&L, shift rota)
│       │   │   ├── tracking/                    # LiveTrackingScreen (Hyderabad Canvas map & pins)
│       │   │   ├── officeregister/              # OfficeRegisterScreen (Digital duty book, cash, fuel)
│       │   │   ├── expenses/                    # ExpensesScreen (Donut chart), AddFuelExpenseScreen
│       │   │   ├── maintenance/                 # MaintenanceScreen (Expiry alert), AddServiceScreen
│       │   │   ├── vault/                       # DocumentVaultScreen (Digital RC, Insurance, PUC cards)
│       │   │   ├── reports/                     # ReportsScreen (Revenue, expenses, driver payout analytics)
│       │   │   ├── alerts/                      # AlertsScreen (Service due, insurance expiry notifications)
│       │   │   ├── settings/                    # SettingsScreen (Owner Master PIN, battery, cloud sync)
│       │   │   └── more/                        # MoreScreen (Fleet operations menu directory)
│       │   ├── components/                      # Reusable UI (GlassCard, PlatformChip, ConfidenceBadge)
│       │   └── theme/                           # Theme.kt, Color.kt (#5B36F5), Shape.kt, Type.kt
│       │
│       ├── util/                                # Helpers & Utility Engines
│       │   ├── StatementExportHelper.kt         # Native A4 PDF & RFC4180 CSV export via FileProvider
│       │   ├── SvmCommissionCalculator.kt       # Commission base, fuel allowance, and settlement math
│       │   ├── Fast2SmsHelper.kt                # SMS gateway client for OTP dispatch
│       │   ├── WhatsAppHelper.kt                # WhatsApp dispatch for driver settlement slips
│       │   ├── AppUpdateHelper.kt               # Self-hosted GitHub APK release updater
│       │   └── BatteryOptimizationHelper.kt     # Unrestricted background run permission launcher
│       │
│       └── work/                                # Background Workers (AndroidX WorkManager)
│           ├── OutboxSyncWorker.kt              # Periodic background flush of offline rides
│           ├── DeviceHeartbeatWorker.kt         # Hourly device health, battery, and status heartbeat
│           └── WorkManagerScheduler.kt          # Worker lifecycle initializer
│
├── .github/workflows/build-apk.yml             # Automated GitHub CI pipeline to build debug APK
├── supabase/migrations/                         # SQL Schemas (005_svm_fleet_office_register.sql)
├── USER_MANUAL.md                               # Complete 200+ line field operational user guide
├── build.gradle.kts & settings.gradle.kts       # Kotlin DSL configuration (Gradle 8.11.1, Kotlin 2.0.20)
└── gradlew & gradlew.bat                        # Bundled Gradle wrapper executables
```

4. Core Navigation Routes & UI Screen Directory

Screen / Route	Role	Primary Composable	Key Features & Capabilities
intro_splash	Public	IntroSplashScreen.kt	Deep Night gradient (#130A3A), road perspective lines, 3-dot pa
auth_login	Public	LoginScreen.kt	Owner phone + Secret Master PIN, Driver phone verification, Fa
owner_dashboard	Owner	DashboardScreen.kt	Total Cars (30), Active (28), Inactive (2), Today's ■18,420 (+12%
owner_cars	Owner	CarsListScreen.kt	Search, filter pills (All/Active/Idle), photo upload, AddCarDialog w
owner_vehicle_detail/{id}	Owner	VehicleDetailScreen.kt	4 Tabs (Overview, Trips, Earnings, Service), Driver Shift Rotatio
owner_live_tracking	Owner	LiveTrackingScreen.kt	Hyderabad vector canvas map, animated car pins, radar pulses,
owner_officeregister	Owner	OfficeRegisterScreen.kt	Digital Moosapet logbook, opening/closing km, fuel deduction, da
owner_expenses	Owner	ExpensesScreen.kt	Interactive donut chart (Fuel 33%, Driver Payouts 54%, Maintena
owner_add_fuel_expense	Owner	AddFuelExpenseScreen.kt	Dedicated form: Car dropdown, fuel type chips, liters, amount ■,
owner_maintenance	Owner	MaintenanceScreen.kt	Upcoming vs History service tabs, insurance/PUC expiry countd
owner_add_service	Owner	AddServiceScreen.kt	Dedicated form: Car dropdown, service type dropdown, date, am
owner_document_vault	Owner	DocumentVaultScreen.kt	Digital compliance cards (RC, Commercial Insurance, PUC, Fitn
owner_reports	Owner	ReportsScreen.kt	Financial statements, car utilization, one-tap export to native A4
owner_settings	Owner	SettingsScreen.kt	Owner Secret Master PIN dialog, 24/7 background exemption, li
driver_today	Driver	DriverTodayScreen.kt	Shift check-in/out, live shift earnings, aggregator ride counter, pl
driver_setup	Driver	SetupWizardScreen.kt	3-step mandatory wizard: Battery exemption, Accessibility permi

5. Production Readiness & Build Verification Summary

✓ **Gradle Build Health:** Verified clean compilation using Gradle 8.11.1 and Kotlin 2.0.20 on Adoptium OpenJDK 17 with **BUILD SUCCESSFUL (zero errors)**.

✓ **Automated Continuous Integration:** GitHub Actions workflow (.github/workflows/build-apk.yml) builds universal debug APK on every push to main.

✓ **Play Store Readiness:** Architecture is modularized into feature components ready for release bundle generation (.aab) and DPDP compliant privacy declarations.